Name: Syed Naziya Parveen

Hall Ticket: 2303A52394

Batch: 42

Task 1: Initial API Setup for a Service Platform

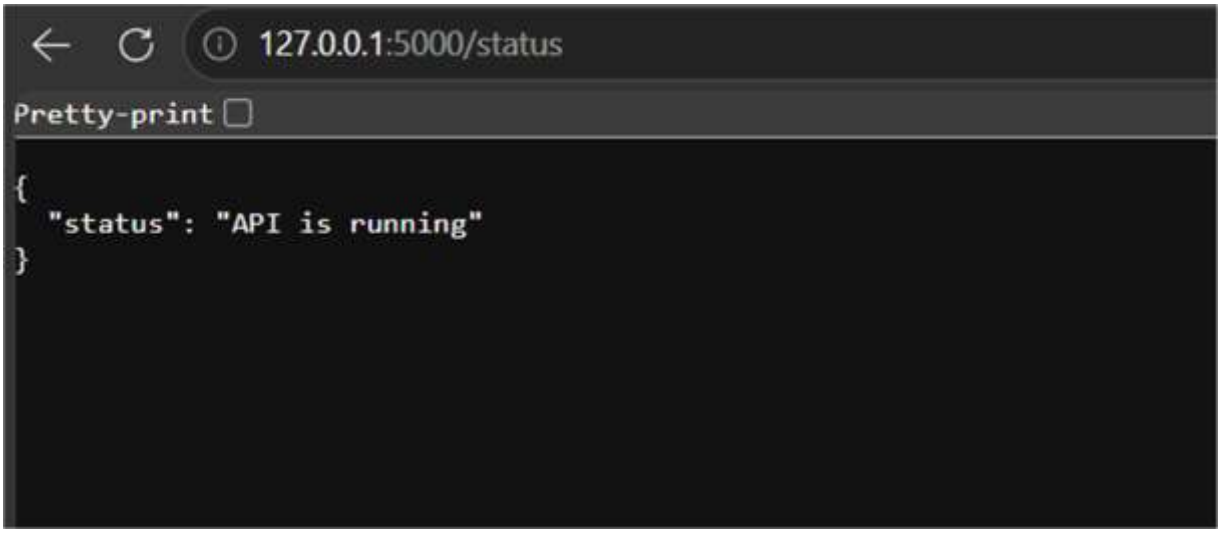
Prompt:

Generate a simple Flask API with a /status endpoint that returns JSON {"status": "API is running"}.
Include code to run the server locally with debug mode enabled.

Code:

```
#Generate a simple Flask API with a /status endpoint that returns JSON {"status": "API is running"}.Include code to run the server locally with debug mode enabled.

from flask import Flask, jsonify app =
Flask(__name__)

@app.route('/status', methods=['GET']) def get_status():
   return jsonify({"status": "API is running"})
 if __name__ == '__main__':
app.run(debug=True)
```

Output:



Justification:

This Flask app creates a basic backend server with one endpoint /status to confirm the API is working. When accessed, it returns a JSON response indica ng the server is running successfully.

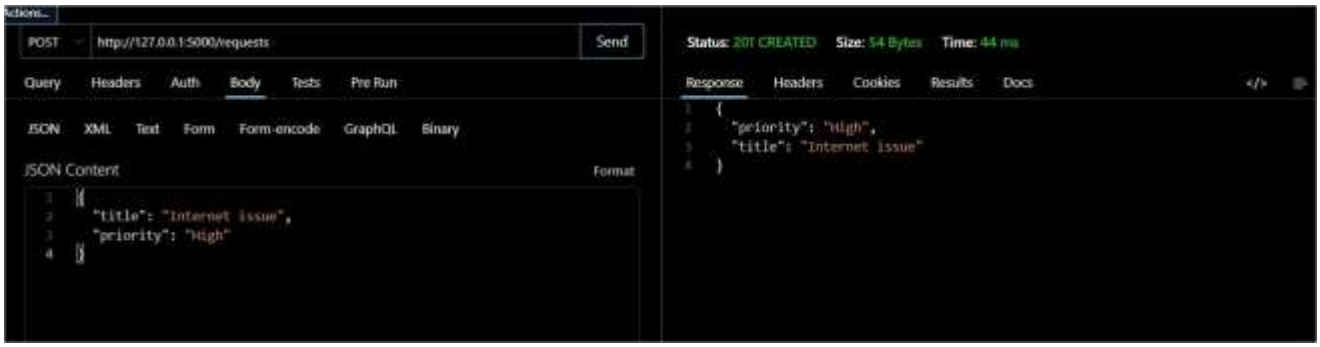
Task 2: Create API to Add and View Resources

Prompt:

Generate Flask API endpoints: GET /requests to return all requests and POST /requests to add a new request with title and priority.Store data in memory (list) and return JSON responses.

Code:

```
#Generate Flask API endpoints: GET /requests to return all requests and POST /requests to add a new request with title and priority.Store data in memory (list) and return JSON responses.
from flask import Flask, request, jsonify app =
Flask(__name__) requests = []

@app.route('/requests', methods=['GET']) def
get_requests():
    return jsonify(requests)

@app.route('/requests', methods=['POST']) def
add_request():     data = request.get_json()
request_item = {
        "title": data.get("title"),
        "priority": data.get("priority")
    }     requests.append(request_item)     return
jsonify(request_item), 201
 if __name__ == '__main__':
app.run(debug=True)
```



Output:

```
Pretty-p  Refresh (Ctrl+R)
[
  {
    "priority": "High",
    "title": "Internet issue"
  }
]
```

Justification

This API stores service requests in a temporary list and allows adding and viewing them using POST and GET endpoints. Data is not permanent restaring the server will erase all stored requests.

Task 3: Modify Exis ng Resource Using AI Assistance

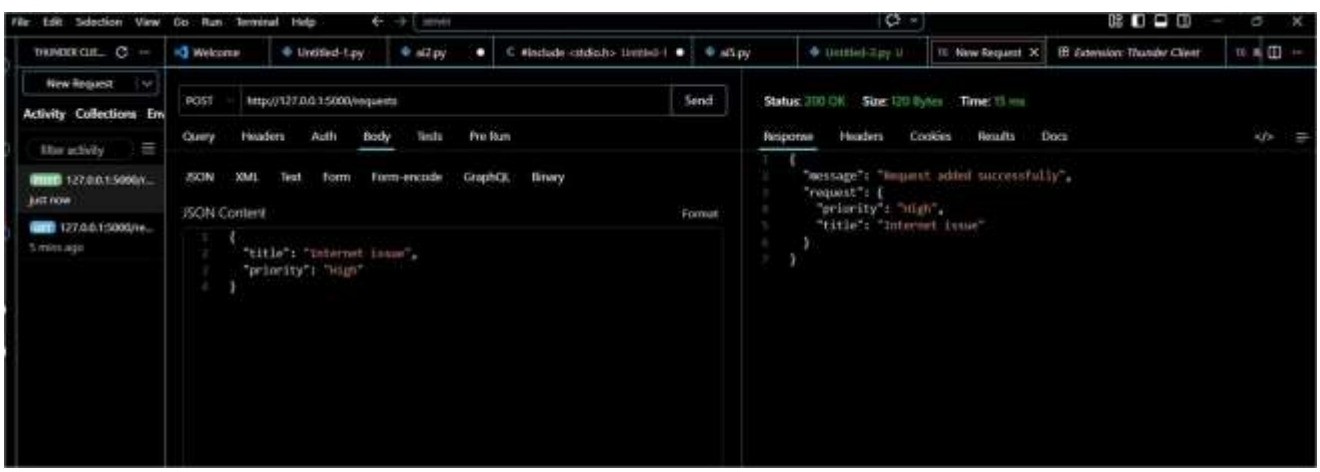
Prompt:

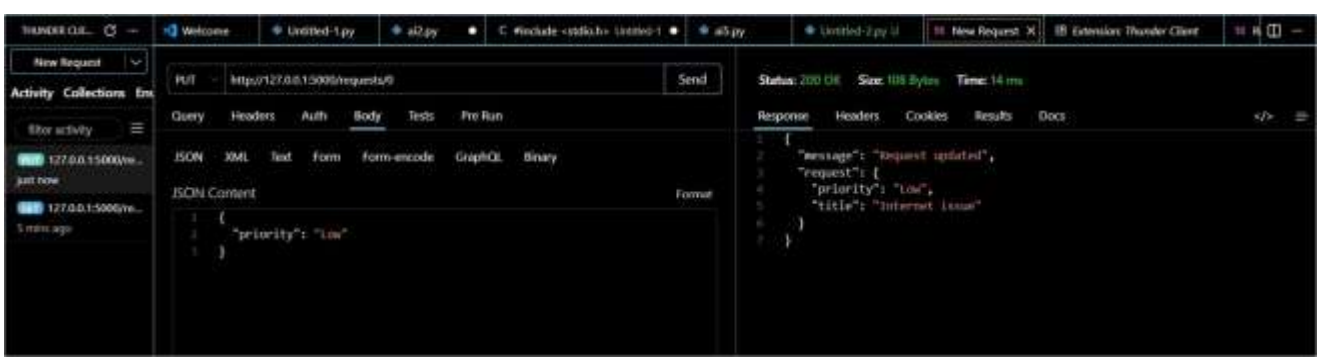
Generate a Flask PUT endpoint /requests/<int:index> to update a request's tle and priority by index.
Return updated data on success and a 404 error if the index is invalid.

Code:

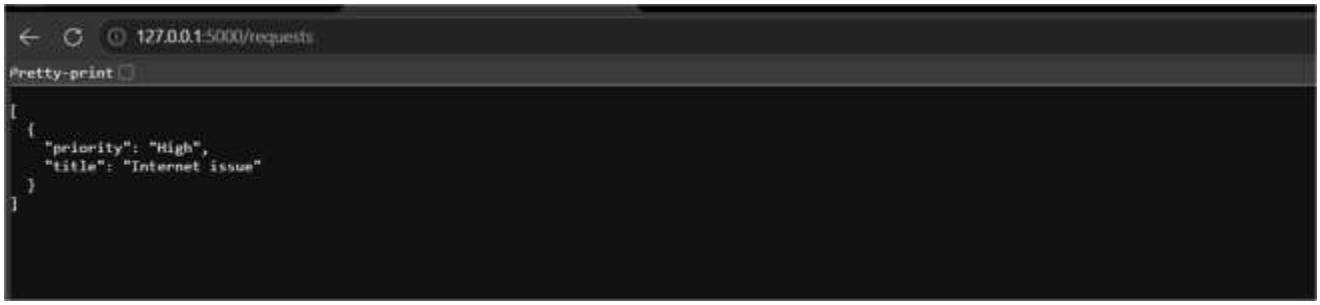
```
#Generate a Flask PUT endpoint /requests/<int:index> to update a request's title and priority by index.Return updated
data on success and a 404 error if the index is invalid.
from flask import Flask, request, jsonify
 app = Flask(__name__)

# In-memory storage requests = []

# STATUS
@app.route('/status') def status():
   return jsonify({"status": "API is running"})

# GET all requests
@app.route('/requests', methods=['GET']) def
get_requests():
   return jsonify(requests)

# POST new request
@app.route('/requests', methods=['POST']) def
add_request():
   data = request.get_json()

   new_request = {
      "title": data.get("title"),
```

```
        "priority": data.get("priority")
    }      requests.append(new_request)

    return jsonify({
        "message": "Request added successfully",
        "request": new_request
    })

# PUT update request
@app.route('/requests/<int:index>', methods=['PUT']) def
update_request(index):     if index < 0 or index >= len(requests):
        return jsonify({"error": "Request not found"}), 404
     data = request.get_json()
     requests[index]["title"] = data.get("title", requests[index]["title"])
    requests[index]["priority"] = data.get("priority", requests[index]["priority"])
     return jsonify({
        "message": "Request updated",
        "request": requests[index]
    })  if __name__ == '__main__':
app.run(debug=True)
```





Output:



Justification:

This endpoint updates an existng request using its index in the list and modifies only provided fields.
It validates the index and returns a 404 error if the request does not exist.

Task 4: Remove a Resource Safely

Prompt:

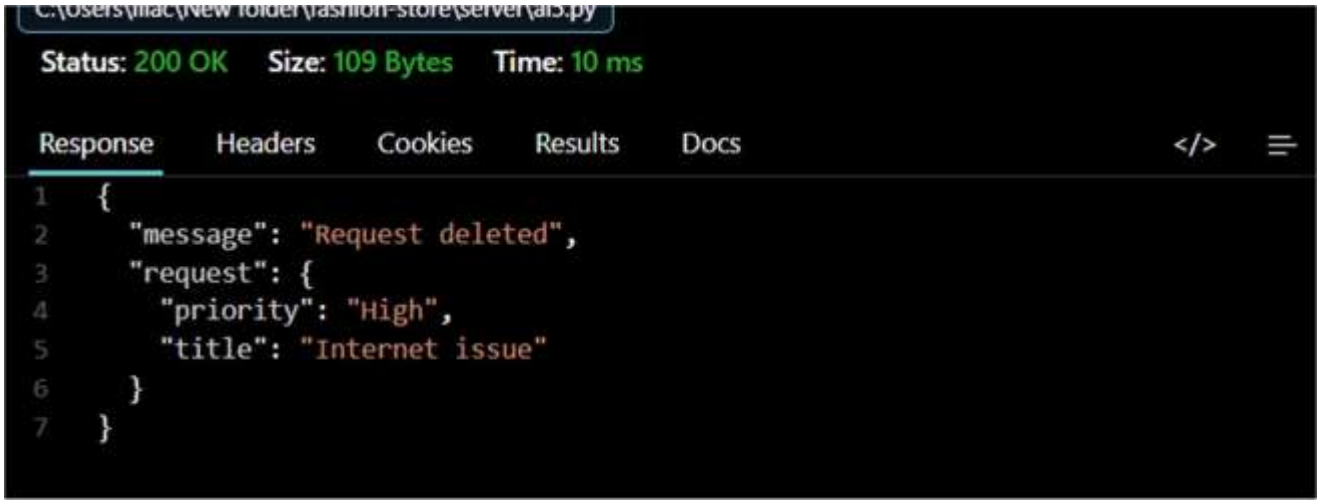
Generate a Flask DELETE endpoint /requests/<int:index> to remove a request by index and return the deleted object.
Handle invalid indexes safely and return a 404 error with a clear message.

Code:

```
from flask import Flask, request, jsonify
 app = Flask(__name__)

# In-memory storage requests = []

# STATUS
@app.route('/status') def status():
   return jsonify({"status": "API is running"})

# GET all requests
@app.route('/requests', methods=['GET']) def
get_requests():
   return jsonify(requests)
```

```
# POST new request
@app.route('/requests', methods=['POST']) def
add_request():
    data = request.get_json()
     new_request = {
        "title": data.get("title"),
        "priority": data.get("priority")
    }       requests.append(new_request)
     return jsonify({
        "message": "Request added successfully",
        "request": new_request
    })

# PUT update request
@app.route('/requests/<int:index>', methods=['PUT']) def
update_request(index):
    if index < 0 or index >= len(requests):
        return jsonify({"error": "Request not found"}), 404

    data = request.get_json()
     requests[index]["title"] = data.get("title", requests[index]["title"])
    requests[index]["priority"] = data.get("priority", requests[index]["priority"])

    return jsonify({
        "message": "Request updated",
        "request": requests[index]
    })
@app.route('/requests/<int:index>', methods=['DELETE']) def
delete_request(index):     if index < 0 or index >= len(requests):
        return jsonify({"error": "Request not found"}), 404
     deleted_request = requests.pop(index)

    return jsonify({
        "message": "Request deleted",
        "request": deleted_request
    })  if __name__ == '__main__':
app.run(debug=True)
```

Output:



Justification

This endpoint removes a request from the list using its index and returns the deleted data.
It prevents crashes by validating the index and returning a 404 error if the request does not exist.

Task 5: AI-Assisted API Documenta on and Readability

Prompt:

Add detailed docstrings and inline comments to all Flask API endpoints describing method, inputs, and responses clearly. Optionally integrate Swagger documenta on using Flask-RESTX to auto-generate API docs UI.

Code:

```
#Add detailed docstrings and inline comments to all Flask API endpoints describing method, inputs, and responses clearly.
Optionally integrate Swagger documentation using Flask-
RESTX to auto-generate API docs UI. from flask
import Flask, request
from flask_restx import Api, Resource, fields
 app = Flask(__name__)

# Swagger UI setup
```

```
api = Api(app, doc='/docs', title='Service API', description='Service Request Management
API')

# Namespace ns = api.namespace('requests', description='Service Requests Operations')
# In-memory storage requests = []

# Model (for Swagger validation + structure) request_model =
api.model('Request', {
    'title': fields.String(required=True, description='Title of the request'),
    'priority': fields.String(required=True, description='Priority level
(High/Medium/Low)')
})

@api.route('/status') class
Status(Resource):     def get(self):
        """Check API status"""         return {"status": "API is
running"}

@ns.route('/') class
RequestList(Resource):
     def get(self):
        """
        GET /requests
        Retrieve all service requests
        """         return requests

    @ns.expect(request_model)     def
post(self):
        """
        POST /requests
        Add a new service request
        """         data = request.get_json()

        new_request = {
            "title": data['title'],
            "priority": data['priority']
        }

        requests.append(new_request)

        return {
            "message": "Request added successfully",
            "request": new_request
        }
```

```
@ns.route('/<int:index>') class
RequestItem(Resource):

   @ns.expect(request_model)     def
put(self, index):
        """
        PUT /requests/<index>
        Update an existing request
        """         if index < 0 or index >= len(requests):
            return {"error": "Request not found"}, 404
         data = request.get_json()

        requests[index]["title"] = data.get("title", requests[index]["title"])
        requests[index]["priority"] = data.get("priority", requests[index]["priority"])
         return {
            "message": "Request updated",
            "request": requests[index]
        }
    def delete(self, index):
        """
        DELETE /requests/<index>
        Delete a service request
        """         if index < 0 or index >= len(requests):
            return {"error": "Request not found"}, 404
         deleted_request = requests.pop(index)
         return {
            "message": "Request deleted",
            "request": deleted_request
        }


if __name__ == '__main__':     app.run(debug=True)
```

Output:

Justification:

This Flask application uses Flask-RESTX to implement CRUD opera ons with automa c Swagger documenta on and input valida on. It provides an interac ve UI where developers can test endpoints and understand request/response formats without reading the code.